Threadplane

From Prototype to Production

Architecture, evaluation and operations for dependable agent products.

Threadplane · Open source · Angular + React

Engineering guide · 2026 · Research reviewed November 7

threadplane.ai

Inside this guide

Practical boundaries, current APIs, and decisions to make before you ship.

01 Start with a bounded job

02 Put ownership at the architecture boundaries

03 Select models with representative evidence

04 Treat context as a governed working set

05 Execute tools through business contracts

06 Recover durable work without inventing certainty

07 Release against observable acceptance criteria

08 Dated capability and reference notes

For senior engineers and technical leaders

Framework-neutral architecture, with Angular and React examples. Published Angular adapters and native source-build bindings are distinct profiles; each example identifies its scope. Native packages remain private and are not npm installation instructions.

Research and repository API review: November 7, 2026. Model catalogs and upstream specifications are dated observations, not a promise of Threadplane support. Threadplane is free and open source; you own your backend, data and deployment.

Start with a bounded job

An agent becomes a production system when people can rely on its outcomes, understand its limits, and recover its work. Better answers help, but they do not settle ownership, authorization, durability, or product economics. Those concerns belong to the application around the model. This paper develops an architecture for that application, with Angular and React as equally important presentation environments.

The running example is hypothetical: a finance analyst investigates a suspected duplicate invoice. The assistant may search authorized records, compare line items, and assemble a proposed credit. A human must approve the exact proposal before a finance service issues it. The amounts, identifiers, and operating targets in this series are illustrative, not customer results or benchmarks. The scenario deliberately combines uncertain interpretation with a consequential, deterministic action.

Choose the smallest useful autonomy

The first decision is whether the job needs an agent. If two invoices share a supplier, invoice number, currency, and total, a deterministic duplicate rule may be enough. If records contain inconsistent descriptions and the analyst needs to investigate several explanations, a bounded agent can contribute judgment. The final credit still belongs behind a conventional service contract.

Work shape	Appropriate starting point	Evidence needed to add autonomy
Known inputs and stable rules	Deterministic workflow	Exceptions require interpretation that rules cannot handle economically
Variable evidence, bounded outcome	One agent with limited tools	Representative tasks show useful investigation and reliable stopping
Independent specialist investigations	Coordinated workers with explicit budgets	Parallel work improves outcomes enough to justify reconciliation
Irreversible business action	Authorized service operation	Proposal validation and approval remain enforceable outside the model

Avoid creating multiple agents merely to mirror an organization chart. Every additional worker adds context transfer, failure handling, and a need to resolve disagreement. For the invoice example, one investigator with read-only tools is a reasonable baseline. A separate agent might later examine contract terms, but only if evaluation shows a benefit over an ordinary retrieval step.

Define success before selecting a model

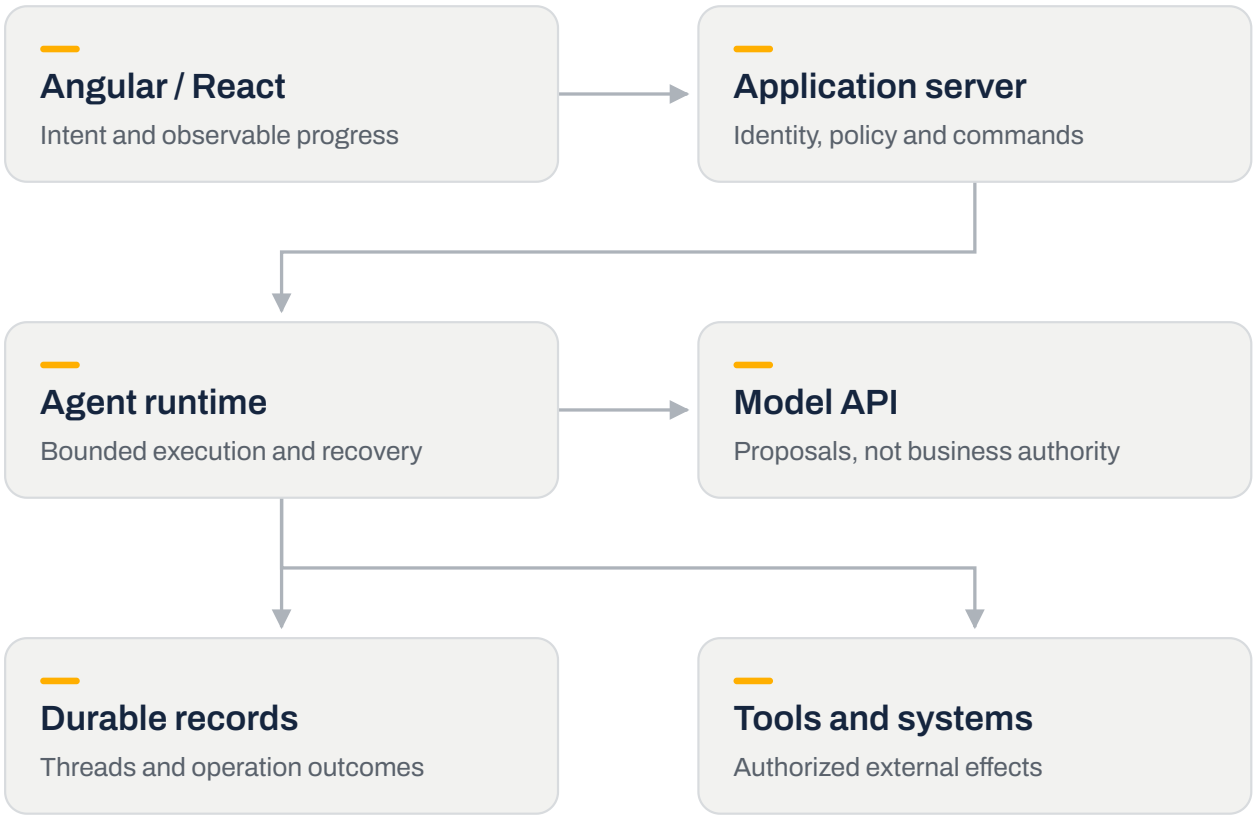
Write a task contract in business terms. A successful investigation identifies the relevant invoices, cites the records used, distinguishes suspicion from confirmation, and produces either a bounded proposal or a reason it cannot proceed. A successful action produces a receipt from the finance service. Fluent explanation without those artifacts does not satisfy the contract.

This separation supports a clear stopping condition. An investigator that cannot access the purchase order should ask for the missing evidence or report the limitation; it should not spend an unlimited tool budget searching increasingly unrelated material. Workflow and agent distinctions are useful starting points, but production boundaries must come from the job itself. Primary reading: Anthropic, Building effective agents.

Put ownership at the architecture boundaries

The model proposes a next step. An execution loop selects permitted actions and interprets results. A durable session records progress. Tools read or change external systems. An application server enforces the user's authority. The interface presents observations and collects intent. These responsibilities can share a process, but they should not disappear into one undifferentiated agent object.

Own the boundaries



The application authorizes intent; the runtime consults models and invokes tools. Models propose actions; runtime and server policy control execution. Durable records survive the view.

In the invoice workspace, the browser sends an investigation request through an authenticated application boundary. The runtime receives a scoped identity and permitted tools. Retrieval supplies invoice evidence; the model proposes a credit; a pending decision is persisted. Approval later authorizes the finance service to attempt the operation. The browser is never the sole record of the proposal or the final result.

Give each lifetime an owner

A conversation may outlive a tab. A run may outlive a connection. A component may mount several times while observing the same run. These are separate lifetimes. Angular and React should render current observations without accidentally acquiring ownership of all three.

Threadplane's native Angular observer returns a signal from an application-owned session. Its native React observer reads the session through an external-store subscription. Destroying either observer releases that subscription; it does not dispose the session. The application decides when to stop work, retain a session across navigation, or release its resources. Published Angular adapters use a different provider-based API, so implementation profiles must remain explicit.

See the local Angular introduction and React introduction for the supported entry points. The broader integration argument is developed in the companion paper, Connecting Agents to Applications.

Buy infrastructure without outsourcing accountability

A managed runtime may supply storage, scheduling, and execution isolation. A self-hosted runtime gives the team more direct control over those mechanisms and their operating burden. Neither choice decides which user may read an invoice, how a stale approval is rejected, or what counts as a completed credit. Procurement should examine those boundaries explicitly.

Ask where state lives, what survives a process restart, how credentials reach tools, what operators can inspect, and how an incident is contained. A vendor's separation of session storage, execution environment, and agent loop can inform this review, but it is a case study rather than a guarantee about another deployment. Primary reading: Anthropic, Managed agents.

Select models with representative evidence

Model selection is a constrained engineering experiment. Start with the task contract and a set of representative investigations: obvious duplicates, legitimate recurring invoices, conflicting currencies, missing purchase orders, inaccessible attachments, and deliberately misleading text inside a document. Include cases where the correct response is to decline a conclusion.

Measure the whole attempt

Compare task completion, evidence quality, tool correctness, latency distribution, and total cost per successful investigation. An inexpensive call that repeatedly chooses the wrong search or requires manual repair may produce an expensive workflow. A stronger model that spends excessive time on an obvious duplicate may also be the wrong default.

Count the entire attempt: retrieval, model input and output, retries, evaluation, and human review. Distinguish time to first visible progress from time to a usable proposal and time to confirmed execution. Streaming can improve perceived responsiveness without making the underlying task finish sooner.

Keep the comparison reproducible. Record the exact model identifier, API, prompt revision, tool catalog, context policy, settings, and evaluation set. Run enough repetitions to expose variable behavior. Report sample size and uncertainty; a handful of attractive transcripts cannot support a universal ranking.

Route only when the route can be evaluated

A practical design might use a lower-latency model to classify incoming requests and a more capable configuration for ambiguous reconciliation. Escalation should follow observable signals such as missing evidence or failed validation, not a free-form claim that the model is confident. The classifier itself needs evaluation, because a mistaken cheap route can hide difficult work from the stronger path.

Reasoning budgets, prompt caching, deferred tool discovery, and programmatic tool composition can change economics, but support varies by provider and model. Treat them as separately measured capabilities. Cache reusable instructions where supported; do not let a cache design turn a user's private evidence into shared application context. Tool discovery can reduce prompt size while introducing a new failure mode: the agent may fail to discover the tool it needs.

Verify the API, not just the name

Changing a model string can leave an old integration semantically wrong. The provider may require a different API for tool calling, expose different modalities, or change lifecycle status. The dated appendix records representative catalog observations; it does not recommend a universal winner or quote prices. Recheck the provider's current contract before deploying a candidate. Primary readings: OpenAI model catalog, reasoning guide, and tools guide.

Treat context as a governed working set

An agent's context is the evidence and instructions available for its next decision. It is not an authoritative database. A long context window can hold more material, but does not decide which material is relevant, current, or permitted. For the invoice investigation, the most useful context may be a few precise records with stable references rather than an entire accounting export.

Retrieve from the systems of record

Retrieve within the current user's access scope. Record which source, version, and retrieval time support each important fact. Separate invoice content from instructions: a note embedded in a supplier attachment is evidence to interpret, not permission to change the assistant's rules or call an unrelated tool.

A retrieval pipeline should be evaluated at two boundaries. First, can it find the necessary records? Second, does the agent use those records correctly? If the relevant purchase order never enters the working set, improving prose generation will not fix the result. If the order is present but misread, changing search ranking may not help.

For example, two invoice totals may match while their service periods differ. A useful investigator checks both periods, extracts the purchase-order reference, and identifies whether a credit already exists. It should preserve conflicting evidence long enough to explain why the proposed duplicate is uncertain. A summary that collapses both invoices into “same amount” has discarded the very distinction needed for the decision.

Make compaction accountable

Long investigations need a working-memory policy. Compact repeated observations and irrelevant detail, but retain unresolved questions, source references, pending decisions, and operation identifiers. Treat generated working notes as fallible summaries. A later step should be able to reopen the original record before committing to a consequential proposal.

Longer-lived memory needs its own purpose and retention policy. A user's preferred table layout is different from a claim that a supplier routinely duplicates invoices. The latter may be sensitive, outdated, or simply wrong. Do not silently promote an agent's tentative conclusion into durable organizational knowledge.

Angular and React clients should receive the information needed for the user task, not every hidden retrieval artifact. A compact evidence panel can show the relevant invoice versions and access-controlled links. The backend retains responsibility for permission checks each time those records are loaded. Context engineering and retrieval guidance inform these mechanisms; the exact retention and evidence policy remains an application decision. Primary readings: Anthropic, Effective context engineering, and LangChain, Deep Agents retrieval.

Execute tools through business contracts

A tool call is an untrusted request until the execution boundary accepts it. A schema can establish that a proposed amount is numeric and an invoice identifier is present. It cannot establish that the invoice belongs to this account, that the amount is appropriate, or that the user may issue the credit.

Validate meaning as well as shape

Split the investigation tools from the action tool. Searching invoices and comparing records can be read-only. Issuing a credit requires current authorization, an approved proposal version, and a business operation identifier. The action service should recompute or verify material facts against authoritative records instead of trusting the model's copied values.

Consider a proposal to credit an illustrative USD 480.00 against invoice INV-1042. Approval must bind to that invoice, currency, amount, account, and proposal revision. If another operator has already credited the invoice, the old approval should not authorize a second credit. The service may reject it or return the previously established outcome, according to the operation contract.

Boundary	Required check	Failure response
Tool request	Shape, supported tool, limits	Reject malformed or unsupported request
Application identity	User and tenant authority	Deny without exposing another tenant's data
Proposal	Current version and business invariants	Refresh evidence and request a new decision
Execution	Idempotency key and operation state	Return existing outcome or a bounded pending state
Result	Receipt and authoritative status	Preserve uncertainty until reconciliation succeeds

Bound the loop

Give tools deadlines and classify failures. A transient read failure may permit a limited retry. A permission denial should not trigger increasingly broad attempts. An uncertain write outcome requires reconciliation before replay. Retry policies should be specific enough that an operator can predict the maximum work and cost of a failed investigation.

Parallel reads can help when they are independent. Parallel writes to the same business object create different risks and require explicit coordination. Tool arguments can also arrive as partial text in a stream; displaying them for progress does not make them executable. Wait for a complete, validated request at the execution boundary.

Structured-output and function-calling facilities help form machine-readable requests. They remain subordinate to the application contract. The model's ability to produce a valid object is useful evidence about formatting, not authorization or business correctness. Primary readings: Anthropic structured outputs and Google Gemini function calling.

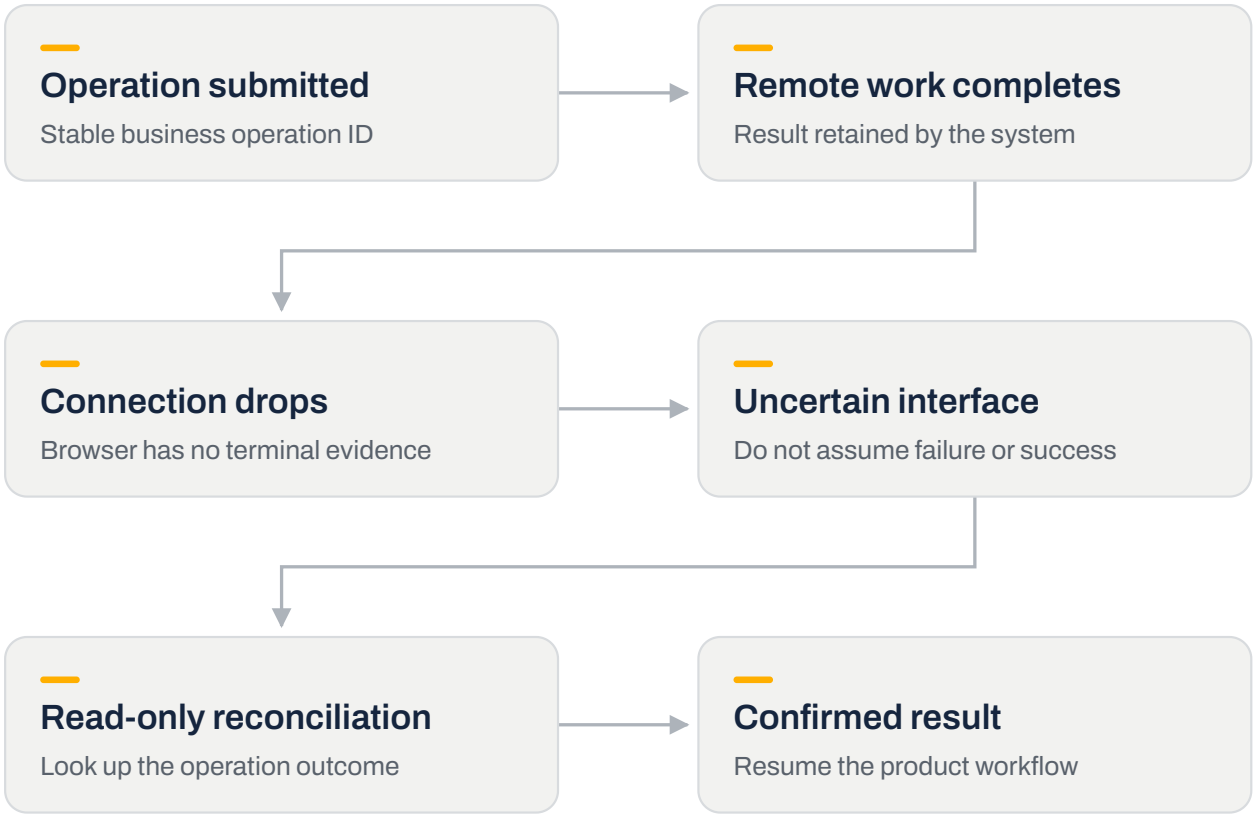
Recover durable work without inventing certainty

Persistence is more than remembering a thread identifier. A recoverable product needs durable runtime state, authorized access to that state, stable business-operation records, and a client that knows how to reconcile them. These pieces answer different questions: what was the agent doing, which conversation is this, and did the external action actually happen?

Checkpoints do not make external effects atomic

A runtime checkpoint can preserve the pending proposal and the next step. It does not automatically make a credit-service write atomic with that checkpoint. A process can fail after the credit succeeds but before the runtime records its receipt. Replaying the last step without a business-level idempotency contract could issue another credit.

Recover without repeating the action



A lost response does not prove the action failed. Reconciliation reads existing evidence. If the outcome remains unknown, preserve that state instead of automatically replaying the command.

In the worked failure case, the analyst approves proposal P-7. The finance service commits operation OP-23, then the connection drops before the browser receives confirmation. The interface shows that the outcome needs checking. A status request resolves OP-23 and obtains the existing credit receipt. The recovered conversation can then explain completion without submitting a second action.

If the status service is unavailable, the product should preserve the unknown state and offer a later check or a supported escalation path. Labeling the action failed would be unjustified; labeling it complete would also be unjustified. Uncertainty is a real domain state, not a rendering defect to conceal.

Separate stopping, reconnecting, and retrying

Stopping a browser stream may only stop observation. A remote run may need a separate cancellation request, and an already committed action cannot be canceled by aborting a fetch. Reconnection reattaches observation where supported. Reconciliation reads authoritative status. Retry intentionally attempts work again. Each command needs its own preconditions and user-facing meaning.

The same distinctions apply in Angular and React. Unmounting a transcript should not silently become a business cancellation policy. A router can restore a thread address, but the server must authorize the load, and the application must distinguish an expired thread from an empty one. Threadplane's Angular persistence guide and React persistence guide describe their respective implementation surfaces.

LangGraph persistence and interrupt documentation provide runtime-specific foundations. Verify the actual deployment's checkpointer, retention, and resume behavior; neither an in-memory demonstration nor a successful refresh proves recovery after process loss. Primary readings: LangGraph persistence and interrupts.

Release against observable acceptance criteria

A production release needs several kinds of evidence. Deterministic UI tests establish how a screen behaves for known snapshots. Adapter tests establish event reduction and command translation. Backend integration tests establish authorization, persistence, and tool contracts. Model evaluations establish how reliably the investigator completes representative tasks. Passing one category cannot substitute for the others.

Build an evaluation ladder

Start with exact checks where exactness is appropriate: the credit amount matches the approved proposal; a denied user cannot load a thread; a repeated operation key does not create a second credit. Use calibrated human or model-assisted review for judgments such as whether an explanation accurately reflects conflicting evidence. Audit model-assisted graders against human decisions and retain disagreements for analysis.

Include difficult negative cases. A retrieval result contains malicious instructions. An invoice disappears between investigation and approval. Two people decide the same proposal. A tool returns a plausible but stale result. An Angular route changes during streaming. A React component remounts while the run remains active. The system should exhibit defined behavior in each case.

Illustrative release gates might require every tested unauthorized write to be denied, every injected uncertain outcome to retain a reconciliation path, and no duplicated finance operation under repeated delivery. Numerical latency or quality targets should come from the team's users and risk tolerance. They should not be presented as an industry benchmark simply because they fit a dashboard.

Operate with an incident model

Trace the investigation, proposal, decision, and operation using distinct identifiers. Capture useful timing and failure categories while redacting credentials and unnecessary record contents. Give operators a way to answer “what happened to OP-23?” without reading every token of a conversation.

Release changes in controlled cohorts, compare them against the established task set, and retain a rollback path for prompts, models, and tool contracts. An incident review should identify the failed boundary: retrieval, interpretation, authorization, execution, observation, or presentation. That classification makes corrective work more precise than attributing every failure to the AI.

The production standard is a task that reaches a trustworthy outcome or a legible, recoverable limit. Evaluation and tracing guidance help build the evidence, while acceptance remains a product responsibility. Primary readings: Anthropic, Demystifying evals for AI agents, and LangSmith observability.

Dated capability and reference notes

Research checked on 7 November 2026. These observations are a maintenance baseline, not a promise that all combinations have been tested together. The chapter links identify primary technical sources; the architectural decisions and invoice scenario are this paper's proposed design.

Representative model candidates

The following is a representative shortlist, not a complete catalog. Every row has the same evaluation status: documentation checked on 7 November 2026; no invoice-task benchmark or deployment validation performed for this paper. Token capacities describe provider or model-card limits, not an independently measured useful-context length.

Identifier and interface	Input and output	Deployment, lifecycle, and limits
`gpt-6-astra`; `gpt-6.1-sol` — OpenAI Responses API	Text and image input; text output	Hosted, listed in the current catalog; 1.05M context and 128K maximum output. Function calling requires Responses for these models.
`claude-opus-5-5` — Claude API	Text and image input; text output; tool use	Hosted, current catalog; 1M context and 128K maximum output. The catalog states retirement no sooner than 22 September 2027; deployment-platform availability must be checked separately.
`gemini-3.8-flash` — Gemini API	Text, image, video, audio, and PDF input; text output	Listed as stable; 1,048,576 input tokens and 65,536 output tokens. Audio generation, image generation, and Live API are unsupported on this model.
`Qwen/Qwen3.8-27B` — downloadable weights for a chosen serving stack	Text plus image/video understanding; text generation	Apache-2.0 metadata; Transformers, vLLM, and SGLang compatibility described. Native context is 262,144 tokens; extension requires the model-card configuration. Hosted service is described as coming soon.

The OpenAI API boundary is consequential: its reasoning guide says Chat Completions does not support function calling with Astra or Sol. A model-name replacement in an older tool loop is therefore insufficient. Sources: OpenAI models and reasoning guide.

Anthropic's catalog supplies the Claude identifier, modalities, capacity, and retirement boundary. Google's model page supplies the Gemini stable identifier, modality list, and unsupported capabilities. These are useful screening facts; neither provider's positioning establishes relative performance on the invoice investigation. Sources: Anthropic models and Gemini 3.8 Flash.

Qwen's downloadable artifacts and its described hosted offering are separate deployment choices. The hosted context and built-in tools must not be attributed to a local weights deployment. Self-hosting also requires capacity planning, runtime configuration, monitoring, and an evaluated tool interface. The model card's benchmark claims are not independent validation for this paper. Source: Qwen's model card.

Framework and package baseline

Upstream Angular 22 and React 19.3 were current in the checked release sources. The repository's native test baselines are Angular 21.1.6 and React 19.2.4; peer ranges are not a complete tested matrix. Published-line Angular packages are version 0.3.7. Native core, Angular, and React packages are private at 0.0.0 and use a source-build profile. This paper does not imply a public React npm installation or identical component coverage.